

1. Двойным факториалом числа n называется произведение всех натуральных чисел от 1 до n с шагом 2. То есть $n!! = \begin{cases} 1 \cdot 3 \cdot 5 \cdot \dots \cdot n & \text{если } n - \text{нечетное} \\ 2 \cdot 4 \cdot 6 \cdot \dots \cdot n & \text{если } n - \text{четное} \end{cases}$

Напишите функцию `double_factorial(n)`, которая принимает натуральное число n и возвращает его двойной факториал.

Примеры:

```
double_factorial(5) = 15
double_factorial(6) = 48
double_factorial(1) = 1
double_factorial(7) = 105
```

2. Напишите функцию `is_prime(n)`, которая принимает целое положительное число n и возвращает `True`, если число простое, и `False` в противном случае. Напомним, что простое число — это натуральное число, имеющее ровно два различных делителя: 1 и само себя. Число 1 не является простым.

Примеры работы функции:

```
is_prime(7) = True
is_prime(10) = False
is_prime(1) = False
```

3. Напишите функцию `count_unique_pairs(arr)`, которая принимает список чисел `arr` и возвращает количество пар одинаковых элементов, то есть количество таких пар (i, j) , что $i \neq j$ и $arr[i] = arr[j]$.

Примеры работы функции:

```
count_unique_pairs([1, 2, 3, 1, 2, 1]) = 4
count_unique_pairs([1, 1, 1, 1]) = 6
count_unique_pairs([1, 2, 3, 4]) = 0
```

4. Напишите функцию `list_difference(a, b)`, которая принимает два списка **различных** чисел `a` и `b` и возвращает список чисел, которые есть в первом списке, но отсутствуют во втором. Если таких чисел нет, функция должна вернуть пустой список.

Примеры работы функции:

```
list_difference([1, 2, 3, 4], [2, 4, 6]) = [1, 3]
list_difference([10, 5, 6, 7], [5, 7]) = [10, 6]
list_difference([1, 2, 3], [1, 2, 3, 4]) = []
```

5. Напишите функцию `rotate(matrix)`, которая поворачивает квадратную матрицу на 90° по часовой стрелке. Напомним, что матрица это двумерный список размера $n \times m$. Функция должна возвращать новую матрицу, не изменяя исходную.

Примеры работы функции:

```
rotate([
    [1, 2],
    [3, 4],
    [5, 6]
]) = [
    [5, 3, 1],
    [6, 4, 2]
]
rotate([[57, 179]]) = [
    [57],
    [179]
]
```

6. Напишите функцию `char_frequency(s)`, которая принимает строку и возвращает словарь, где ключи — символы строки, а значения — их количество вхождений.

Примеры работы функции:

```
char_frequency("hello") = {
    'h': 1,
    'e': 1,
    'l': 2,
    'o': 1
}

char_frequency("abracadabra") = {
    'a': 5,
    'b': 2,
    'r': 2,
    'c': 1,
    'd': 1
}
```

7. Напишите функцию `sum_of_two(arr, target)`, которая принимает массив целых чисел `arr` и целое число `target`. Функция должна вернуть индексы двух элементов, сумма которых равна `target`. Если таких элементов нет, верните пустой список `[]`. Индексы должны быть в порядке возрастания.

Ограничения: Алгоритм должен работать за $O(n)$ времени (один проход по массиву).

Примеры работы:

```
sum_of_two([1, 2, 7, 11, 15], 9) = [0, 1]

sum_of_two([3, 2, 4], 6) = [1, 2]

sum_of_two([3, 3], 6) = [0, 1]

sum_of_two([1, 2, 3, 4, 5], 10) = []
```